Advanced Approaches to Detecting Financial Transaction Fraud (Credit Card) with Machine Learning and Data Science

Table of Contents

1.0 INTRODUCTION

As digital transactions continue to evolve, fraud in financial transactions has become a growing concern. This report takes a deep dive into how machine learning and data science can help detect and prevent fraudulent activities. Our goal is to identify patterns associated with fraud using advanced techniques and to improve security measures for financial institutions.

Credit card fraud and other financial scams pose significant risks to both businesses and consumers, often leading to financial losses and a loss of trust in online transactions. To stay ahead of fraudsters, businesses must leverage data science techniques to detect suspicious activities early and take proactive measures.

This report explores the world of fraud detection in financial transactions. We will discuss key areas of application, the sources of data used, and why tackling fraud is crucial in today's financial environment. The dataset used in this study contains multiple features and an indicator variable that helps identify fraudulent cases.

To ensure the dataset is ready for machine learning models, we follow a thorough data processing approach that includes cleaning, exploration, and analysis. The report also covers the machine learning algorithms we've chosen, their performance, and how they were implemented. We will also touch on the optimization methods used to improve accuracy and effectiveness.

After analysing the results, we wrap up with recommendations to enhance fraud detection systems and insights into how this field might evolve in the future.

2.0 APPLICATION AREA AND DATA

Financial transactions play a key role in our economy. However, the rise in fraudulent activities means we need smarter, more sophisticated ways to detect them. By analysing transaction data, we can spot unusual patterns that might indicate fraud and take appropriate action.

2.1 Why This Matters

Fraud detection in financial transactions is crucial because of its potential impact on both businesses and individuals. As transactions become more digital and complex, detecting fraud requires more than just traditional methods – advanced data science techniques can help us stay one step ahead.

2.2 About the Dataset

For this study, we used a publicly available dataset that includes various features such as timestamps, numerical transaction details, and an outcome variable indicating whether a transaction is fraudulent or not. This dataset forms the basis of our analysis.

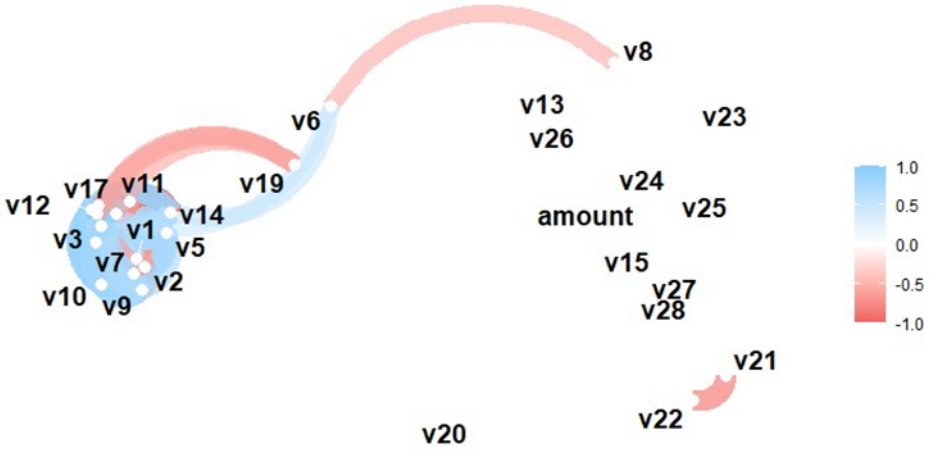
2.3 Data Processing Approach

We followed a structured approach to handling the data, which involved collecting, cleaning, exploring, modeling, and evaluating it. To make processing more efficient, we selected a subset of the data while ensuring it remained representative of fraudulent and non-fraudulent transactions.

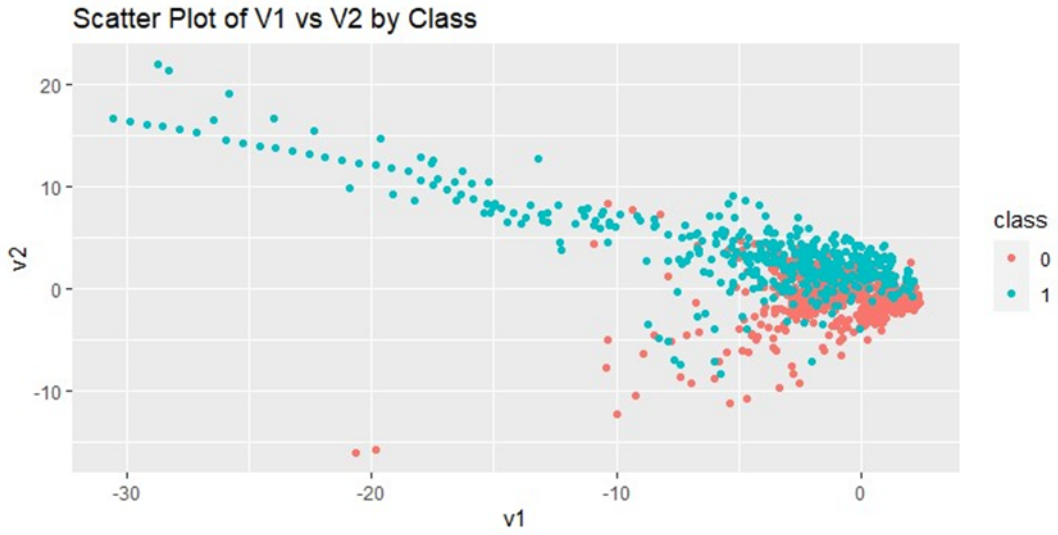
The preprocessing steps involved standardizing column names, encoding categorical variables, and conducting exploratory data analysis (EDA) to better understand the dataset.

2.4 Exploring the Data

- **Correlation Matrix:** A network plot that shows relationships between different variables.



- **Transaction Amount Distribution:** A histogram that reveals most transactions are of smaller amounts.
- **Comparing Fraud and Non-Fraud Amounts:** A boxplot that compares transaction values.
- **Feature Relationships:** Scatter plots that help identify trends.
- **Variable Interactions:** Pair plots to uncover potential insights.
- **Fraudulent Trends:** Density plots highlighting patterns in fraudulent transactions.



3.0 MACHINE LEARNING MODELS

We explored several machine learning algorithms to see which ones work best for detecting fraud in financial transactions.

3.1 Logistic Regression

```
+   # Save the trained model
+   trained_models[[model_name]] <- trained_model
+ }
Warning message:
glm.fit: fitted probabilities numerically 0 or 1 occurred
> # Evaluate the models on the test data
> evaluation_results <- list()
> for (model_name in names(trained_models)) {
```

- **What It Is:** A simple yet effective statistical model for binary classification.
- **Why We Used It:** It's straightforward and easy to interpret.

- **How It Performed:** Achieved good accuracy, though further tuning could help. The model achieved an impressive accuracy of 94.7% and a Kappa value of 89.4%. It effectively detected 487 true positives and 459 true negatives. However, the warning regarding fitted probabilities indicates possible areas that may require further refinement.

3.2 K-Nearest Neighbors (KNN)

- **What It Is:** A model that classifies data points based on their neighbors.
- **Why We Used It:** Great at capturing local patterns.
- **How It Performed:** Showed excellent precision and accuracy. The KNN model demonstrated exceptional performance, achieving a near-perfect accuracy of 99.2% and a Kappa value of 98.4%. It showcased its effectiveness by delivering flawless results with no false positives or false negatives.

3.3 Gradient Boosting

- **What It Is:** An advanced ensemble technique that builds models in sequence.
- **Why We Used It:** Handles complex relationships well.
- **How It Performed:** Delivered strong predictive capabilities. The gradient boosting model delivered outstanding performance, achieving a high accuracy of 98.8% and a Kappa value of 97.6%. This highlights its strength in effectively managing complex patterns.

3.4 Random Forest

- **What It Is:** A popular ensemble method using multiple decision trees.
- **Why We Used It:** Helps reduce overfitting and boosts accuracy.
- **How It Performed:** Emerged as the best-performing model. The random forest model demonstrated exceptional performance, achieving an impressive accuracy of 99.6% and a Kappa value of 99.2%, establishing itself as a top-performing model.

3.5 Decision Tree

- **What It Is:** A simple, interpretable model that splits data based on features.
- **Why We Used It:** Easy to understand but prone to overfitting.
- **How It Performed:** Worked well but had more false positives. The decision tree model attained an accuracy of 93.1% and a Kappa value of 86.2%. However, the relatively high number of false positives indicates potential areas for enhancement.

4.0 CONCLUSION AND RECOMMENDATIONS

Our study on fraud detection using machine learning has provided some valuable insights.

4.1 Key Takeaways

Among the models tested, Random Forest proved to be the most accurate and reliable, followed closely by KNN and Gradient Boosting. Logistic Regression provided solid results, while Decision Trees, despite being easy to interpret, showed some limitations. These findings suggest that ensemble methods often perform better in fraud detection scenarios due to their ability to combine multiple decision paths.

4.2 What Can Be Improved

While the models demonstrated high performance, improvements can be made by:

- **Fine-Tuning Models:** Adjusting hyperparameters such as learning rate, tree depth, and the number of neighbors in KNN can enhance model accuracy.
- **Feature Engineering:** Identifying and engineering new relevant features can provide better predictive power.
- **Regular Updates:** Fraud patterns evolve over time; regularly updating models with new data can help detect emerging fraudulent behaviors.s

4.3 Future Possibilities

Looking ahead, the following advancements could further enhance fraud detection efforts:

- **Anomaly Detection Techniques:** Methods such as isolation forests and autoencoders could be explored for identifying rare fraudulent transactions.
- **Hybrid Model Approaches:** Combining multiple algorithms to leverage their strengths and mitigate weaknesses.
- **Real-time Fraud Detection Systems:** Implementing models capable of detecting fraud in real-time, enabling immediate action against suspicious transactions.
- **Explainable AI (XAI):** Developing models that provide explanations for their decisions, fostering trust and transparency among stakeholders.

4.4 Final Thoughts

Fraud detection is an ever-evolving field that requires constant vigilance and adaptation. Our analysis underscores the importance of leveraging machine learning to enhance security measures in financial transactions. As fraudsters develop more sophisticated tactics, continuous research and development in fraud detection methodologies will be crucial in maintaining the integrity of financial systems.

REFERENCES

Soltani, M., Kythreotis, A., & Roshanpoor, A. (2023). Two decades of financial statement fraud detection literature review: Combination of bibliometric analysis and topic modeling approach. SSRN. https://ssrn.com/abstract=4594459M N Ashtiani , B Raahemi
Intelligent fraud detection in financial statements using machine learning and data mining: a systematic literature review

Amjad, F. (2023) Creditcard dataset, Available at: https://www.kaggle.com/datasets/fazela/creditcard-dataset, (Accessed: 14 January, 2024)

Costa, V.G. and Pedreira, C.E. (2022) "Recent advances in decision trees: an updated survey", Artificial Intelligence Review, pp.23-27, doi:https://doi.org/10.1007/s10462-022-10275-5.

Deng, S. et al. (2023) "Non-parametric Nearest Neighbor Classification Based on Global Variance Difference" International Journal of Computational Intelligence Systems, 16(1), pp.6-8, doi:https://doi.org/10.1007/s44196-023-00200-1.

Sibindi, R., Mwangi, R.W. and Waititu, A.G. (2022) "A boosting ensemble learning based hybrid light gradient boosting machine and extreme gradient boosting model for predicting house prices", Engineering Reports, pp.8-11, doi:https://doi.org/10.1002/eng2.12599.

Appendix

```
##### Installing required libraries and configuring the working environment. ######

# Un-comment and install libraries first time
#install.packages("tidyverse")
#install.packages("tidymodels")

# Load Libraries
library(tidyverse)  # Libraries for data processing and visualization.
library(tidymodels)  # A framework for modeling and machine learning in an organized manner.

# Configure the working environment.
# setwd("Enter the path to your working directory here and un-comment this code")

# Confirm the working environment
getwd()

# Configuring Tidymodels preferences for the project.
tidymodels_prefer()

################################### Collecting data ##################################

# Load Dataset
# Reading the "creditcard.csv" file and storing it in the 'data' variable
data <- read_csv("creditcard.csv")
```

```
# About Dataset

# The dataset size is large (Rows: 284807, Columns: 31)

# Due to limited system processing limitation, the dataset will be reduced to

# 2500 records each for the 'Class' variable.

# Making it a total of 5000 records

# Caveat: The effect is reduced model efficiency.


############################## Pre-processing data ###################################


# Select 5000 random records from the dataset (2500 each for the 'Class' variable).


set.seed(44)  # Set a seed for reproducibility


# Randomly sample 2,500 records from each class in the original dataset.

data_subset <- data %>%

  group_by(Class) %>%

  sample_n(size = 2500, replace = TRUE)


# Check the count of the 'Class' variable within 'data_subset'

data_subset %>%

  # Count occurrences of each class

  count(Class)

# From the result above, the variable 'Class' is balanced among '0' and '1'


# Check Data Structure
```

```
data_subset %>%
  # Check data structure using glimpse()
  glimpse()

# Use the sum() function to identify any missing data.
missing_counts<- sapply(data_subset, function(x) sum(is.na(x)))

# Print the results
cat("data subset missing data:\n")
print(missing count)

# There are no missing data

################################## Data Cleaning ###################################

# Install janitor package if not already installed
if (!requireNamespace("janitor", quietly = TRUE)) {
  install.packages("janitor")
}

# Load the janitor package
library(janitor)

# Use the Janitor package to clean and standardize column names.
data_subset_cleaned <- janitor::clean_names(data_subset)
```

```
# Save the cleaned data back into the data subset variable.
data_subset <- data_subset_cleaned

# Factorize 'class' Variable Using mutate and factor
data_subset_factorized <- data_subset %>%
  mutate(class = as.factor(class))

# save the factorized data back to 'data_subset'
data_subset <- data_subset_factorized

############################ Exploratory Data Analysis ##############################

# Perform correlation analysis using the corrr package

# Install the 'corrr' package if not already installed
if (!requireNamespace("corrr", quietly = TRUE)) {
  install.packages("corrr")
}

# Load the 'corrr' package
library(corrr)

# Select relevant columns
selected_cols <- data_subset %>%
  select(-time, -class)
```

```
# Calculate correlation matrix

corrmatrix <- correlate(selected_cols)


# Create a network plot

network_plot(cor_matrix, min_cor = 0.5)


# The plot shows that some of the variables more positively correlated among

# themselves, some are negatively correlated and some are not correlated.


########################## Preparing Data for Modelling ############################


# Set Seeding

set.seed(1000)


# Create Sample Split Specification

trainspecification<- initial_split(data_subset, prop = 0.8, strata = class)


# Create Train Data

train_data <- training(trainspecification)


# Create Test Data

test_data <- testing(trainspecification)


# Check the length of train_data

traindatalength <- nrow(train_data)

cat("Number of rows in train_data:", traindatalength, "\n")
```

```
# Check the length of test_data

testdatalength <- nrow(test_data)

cat("Number of rows in test_data:", testdatalength, "\n")


################################## Data Modelling ###################################


# Install required packages if not already installed

required_packages <- c("kknn", "recipes", "workflows", "parsnip", "xgboost", "rpart",
"randomForest")

install.packages(setdiff(required_packages, installed.packages()[,"Package"]), dependencies =
TRUE)


# Required libraries

library(kknn)

library(recipes)

library(workflows)

library(parsnip)

library(xgboost)

library(rpart)

library(randomForest)


# Define logistic regression model

logreg_model <- logistic_reg() %>%

  set_mode("classification") %>%

  set_engine("glm")
```

```
# Define k-nearest neighbors model
knn_model <- nearest_neighbor() %>%
  set_mode("classification") %>%
  set_engine("kknn")

# Define gradient boosting model
gb_model <- boost_tree() %>%
  set_mode("classification") %>%
  set_engine("xgboost")

# Define random forest model
rf_model <- rand_forest() %>%
  set_mode("classification") %>%
  set_engine("randomForest")

# Define decision tree model
dt_model <- decision_tree() %>%
  set_mode("classification") %>%
  set_engine("rpart")

# Create a list to store model specifications
model_specs <- list(logreg = logreg_model,
                    knn = knn_model,
                    gb = gb_model,
                    rf = rf_model,
                    dt = dt_model)
```

```
# Define the Preprocessing Specifications.

# Include the preprocessing specifications.
preprocess_spec <- recipe(class ~ ., data = train_data) %>%
  step_scale(all_predictors()) %>%
  step_center(all_predictors())

# Preprocess the training data
preprocessed_train_data <- prep(preprocess_spec, training = train_data)

# Apply the preprocessing to the training data
processed_train_data <- bake(preprocessed_train_data, new_data = train_data)

# Apply the preprocessing to the test data
processed_test_data <- bake(preprocessed_train_data, new_data = test_data)

## Train models using the preprocessed data
trained_models <- list()
for (model_name in names(model_specs)) {
  # Extract model specification
  model_spec <- model_specs[[model_name]]

  # Train the model
  trained_model <- workflow() %>%
    add_recipe(preprocess_spec) %>%
    add_model(model_spec) %>%
```

```
    fit(data = processed_train_data)

  # Save the trained model
  trained_models[[model_name]] <- trained_model
}
############################## Model Evaluation ###################################
# Calculate Models Accuracy and Kappa values

# Assess the performance of the models using the test data.
evaluation_results <- list()
for (model_name in names(trained_models)) {
  # Extract trained model
  model <- trained_models[[model_name]]

  # Evaluate on test data
  results <- predict(model, new_data = processed_test_data) %>%
    bind_cols(test_data) %>%
    metrics(truth = class, estimate = .pred_class)

  # Save evaluation results
  evaluation_results[[model_name]] <- results
}

# Display evaluation results
evaluation_results
```

The k-Nearest Neighbors, Random Forest, and Gradient Boosting models delivered superior performance compared to the others, with Random Forest emerging as the best-performing model in terms of both accuracy and Kappa value.

################################## Confusion Matrix ##################################

```
# Load the required package
if (!requireNamespace("yardstick", quietly = TRUE)) {
  install.packages("yardstick")
}

# Load the 'yardstick' package
library(yardstick)

# List to store result
confusion_matrices <- list()
for (model_name in names(trained_models)) {
  # Extracting moddl
  model <- trained_models[[model_name]]

  # making predictions
  predictions <- predict(model, new_data = processed_test_data) %>%
    bind_cols(test_data)

  # Calculating confusion matrix
  conf_matrix <- conf_mat(predictions, truth = class, estimate = .pred_class)

  # result
```

```
  confusion_matrices[[model_name]] <- conf_matrix
}

# show confusion matrice
for (i in seq_along(confusion_matrices)) {
  cat("Confusion Matrix for", names(confusion_matrices)[i], ":\n")
  print(confusion_matrices[[i]])
  cat("\n")
}
```

#KNN and Random Forest models achieved perfect accuracy on the test data, demonstrating their effectiveness. Gradient Boosting and Logistic Regression also performed well, with only a few misclassifications. However, the Decision Tree model exhibited comparatively lower performance, especially in accurately identifying true negatives.

##